

Chapter 1

Proposed Solution

While Chapter 2 established the problem context and the theoretical building blocks, this chapter describes the proposed solution in detail. Section 1.1 gives a bird’s-eye view of the entire processing pipeline, accompanied by a flow diagram, so that the reader can grasp how a raw log line is transformed into a security verdict. The subsequent sections then zoom into each stage of the pipeline. First, Section 1.2 describes request parsing and the extraction of the shared 22-feature vector. The three detection tiers are then presented one by one: the signature- and rule-based tier (Section 1.3), the supervised Random Forest tier (Section 1.4), and the unsupervised one-class anomaly tier (Section 1.5). Finally, Section 1.6 explains the smart hybrid consensus and the threat-scoring scheme that fuse the three tiers into a single, explainable decision. For each stage, a step-by-step description of the underlying procedure complements—but does not replace—the narrative explanation.

1.1 Overview

The proposed system is a *three-tier hybrid* web intrusion detection pipeline that consumes Apache access logs and emits, for every request, a binary verdict (attack or clean), a continuous threat score, and a severity label. The design principle is to combine three detection paradigms that have complementary strengths and weaknesses, as motivated in the related-work survey of Chapter 2: a high-precision signature tier, a high-accuracy supervised tier, and a high-recall unsupervised tier capable of flagging previously unseen attacks. A consensus mechanism then reconciles the three opinions so that the final decision inherits the recall of anomaly detection while retaining the precision of the supervised model.

Figure 1.1 depicts the end-to-end flow of the solution. The processing of a single log line proceeds through the following main steps, which correspond directly to the boxes in the diagram:

- (i) **Parsing and decoding.** Each log line is parsed into its constituent fields and its URL is iteratively decoded to undo multi-layer encoding.
- (ii) **Feature extraction.** The parsed request is turned into a 22-dimensional numeric feature vector shared by the two machine learning tiers.
- (iii) **Whitelist filtering.** Requests for harmless static resources (images, stylesheets, scripts) are recognised and exempted from the machine learning tiers to reduce noise.

- (iv) **Three-tier detection.** The request is evaluated in parallel by Tier 1 (signature and rules), Tier 2 (Random Forest probability), and Tier 3 (one-class anomaly model).
- (v) **Consensus and scoring.** The three tier outputs are fused by the smart consensus rule into a verdict, and a weighted threat score assigns a severity level.
- (vi) **Alerting.** The structured result is written to disk in JSON Lines format and surfaced on a real-time dashboard.

Diagram to be produced. A top-to-bottom flowchart with a parallel branch in the middle. The recommended layout is:

- *Top (input):* a parallelogram (data node) labelled “Apache access log (one line per request)”.
- A downward arrow to a rectangle “Parse & decode URL”.
- A downward arrow to a rectangle “22-feature extraction”.
- A downward arrow to a rectangle “Static-resource whitelist filter”.
- From this rectangle, *three arrows fan out in parallel* to three side-by-side boxes: “Tier 1: Signature + rules”, “Tier 2: Random Forest (p_{attack})”, and “Tier 3: One-class LOF”.
- The three boxes *converge* with three arrows into a diamond (decision node) labelled “Smart consensus”.
- A downward arrow to a rectangle “Threat score & severity bucket”.
- A downward arrow to a parallelogram (output) “Alert + dashboard (JSONL)”.

Suggested styling: rounded rectangles for processing steps, parallelograms for input/output, a diamond for the consensus decision, and a light grey fill for the three tier boxes to emphasise the parallel stage.

Figure 1.1: End-to-end processing flow of the proposed three-tier hybrid web intrusion detection system.

The overall scanning procedure, implemented in the function “analyze_log”, formalises this flow. It iterates over the log lines, applies the parsing and feature steps, evaluates the three tiers (subject to the whitelist exemption), and then delegates the final verdict to the consensus and scoring procedure detailed in Section 1.6. Concretely, the procedure carries out the following steps for the input log file:

1. For each log line, parse the line into its fields and produce the decoded URL.
2. Extract the 22-dimensional feature vector from the parsed request.
3. If the request is whitelisted (a static asset, or a request with status code 408), skip the two learning tiers and treat their outputs as “not fired”; otherwise continue.
4. Evaluate Tier 1 to obtain the set of fired rule identifiers.
5. Evaluate Tier 2 to obtain the Random Forest attack probability p .
6. Evaluate Tier 3 to obtain the one-class anomaly flag a .

7. Call the consensus procedure with the rule set, p , and a to obtain the verdict, the threat score, and the severity level.
8. Write the structured record (verdict, score, severity, contributing tiers) to the output file.

In addition to the per-request analysis, the system performs a lightweight behavioural pass after the loop. It tracks, for every source IP address, the set of distinct paths requested; an IP that touches at least ten distinct paths is flagged as a probable directory or port scanner. This complements the content-based tiers with a simple traffic-pattern heuristic. The same pipeline is reused unchanged in the real-time monitoring mode, where log lines are read from standard input and alerts are streamed to the dashboard as they occur.

1.2 Request Parsing and Feature Extraction

1.2.1 Parsing and multi-layer URL decoding

The first stage converts a raw Apache log line into a normalised structure used by every downstream component. The function “`parse_log_entry`” relies on a log parser configured for the Combined Log Format, with a fallback to the Common Log Format for lines that lack the “Referer”/“User-Agent” headers. From the request line it extracts the HTTP method, the raw URL, and the protocol; it then URL-decodes the URL and splits it into a path, a query string, and a dictionary of parameters. The status code, the “User-Agent”, and the “Referer” headers are also retained, since they feed several features described below. The result is a record with the fields “path”, “query”, “method”, “status”, “user_agent”, and “referer”, which constitutes the common input contract for all three tiers.

A subtle but important detail is the handling of encoded payloads. As discussed in Chapter 2, attackers hide malicious content behind several layers of URL encoding. The helper “`multi_decode`” therefore decodes the URL repeatedly, up to three iterations, stopping early when a further decoding leaves the string unchanged. This recovers the original payload and, as a by-product, yields the *encoding depth* (the number of layers that were actually peeled off), which is itself an informative feature.

1.2.2 The shared 22-feature vector

Both machine learning tiers operate on the same 22-dimensional feature vector, produced by the “`extract_features`” method. Sharing a single feature space guarantees that the performance gap between the supervised and unsupervised tiers reflects the “labelled versus unlabelled” distinction alone, rather than any difference in representation. The twenty-two features fall into three conceptual groups—structural, randomness-and-encoding, and semantic-and-contextual—which are described feature by feature below; the corresponding source code is shown in Figure 1.2.

Structural group (nine features). This group captures the coarse size and shape of the request. It comprises the total URL length (“`len_url`”), counted over the concatenated path and query; the path depth (“`path_depth`”), the number of “/” separators in

the path, which grows for deeply nested or traversal-style endpoints; a binary POST flag (“is_post”) recording the request method; the parameter count (“param_count”), taken as the number of “=” signs in the query; the raw count of dangerous metacharacters (such as the semicolon, pipe, dollar sign, back-tick, parentheses, angle brackets, quotes, braces, and backslash) in the decoded URL (“raw_risk_count”) together with their density relative to the URL length (“risk_ratio”); the number of SQL, XSS, and LFI keywords found in the decoded URL (“keyword_hits”); the percent-encoding density, i.e. the proportion of “%” characters (“encoding_ratio”); and the Shannon entropy of the decoded URL (“entropy”). For a string s , the Shannon entropy is

$$H(s) = - \sum_{c \in \Sigma} \frac{n_c}{|s|} \log_2 \frac{n_c}{|s|}, \quad (1.1)$$

where Σ is the set of distinct characters occurring in s , n_c is the number of occurrences of character c , and $|s|$ is the total length of s , so that $n_c/|s|$ is the relative frequency of c . A payload that has been encoded or randomised spreads its characters more uniformly and therefore attains a higher entropy, which makes Equation (1.1) a compact, signature-independent signal of obfuscation. Taken together, an over-long URL, a high risky-character density, or an abnormally high entropy are all hallmarks of an injection or buffer-overflow attempt.

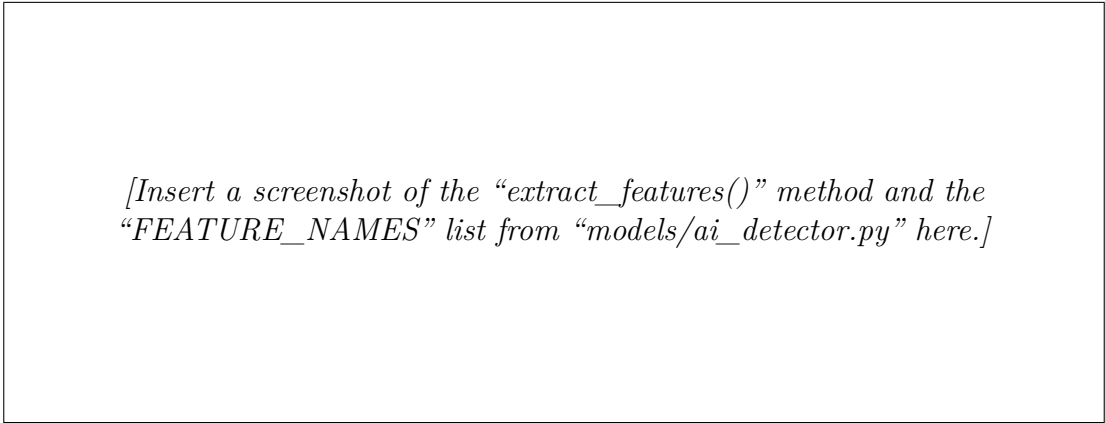
Randomness-and-encoding group (five features). This group quantifies obfuscation and the multi-layer encoding evasion discussed in Chapter 2. It contains the maximum Shannon entropy taken over all individual parameter values (“param_value_max_entropy”), which isolates a single high-entropy field even when the rest of the URL looks ordinary; the length of the longest parameter value (“longest_param_value”), which flags an oversized payload smuggled into one field; the encoding depth (“encoding_depth”), the number of URL-decoding passes that were actually required (from 0 up to 3) before the payload stopped changing, which directly measures how many layers of encoding the attacker stacked; the consecutive-encoding intensity (“consecutive_encoding_intensity”), the fraction of the URL occupied by runs of consecutive “%xx” sequences, which is high for heavily encoded payloads; and the number of parameter names drawn from a curated suspicious set such as “cmd”, “exec”, “file”, or “passwd” (“param_names_anomaly”).

Semantic-and-contextual group (eight features). This group encodes domain knowledge about how attacks differ from legitimate traffic. It comprises the suspicious-User-Agent score (“suspicious_ua_score”), a value in $[0, 1]$ that is maximal for a known scanner string (“sqlmap”, “nikto”, and the like), moderate for an automated client, and high for a missing User-Agent; the Referer-suspicion score (“referer_suspicion”), likewise in $[0, 1]$, which rises when the Referer header itself contains attack tokens or is heavily percent-encoded; the status-code indicator (“status_code_indicator”), a small bucket of the HTTP response status (0 for 2xx, 1 for 3xx, 2 for 4xx, 3 for 5xx) that captures error-inducing requests; the signature score (“signature_score”), the number of Tier-1 attack regular expressions (out of four) that match the decoded payload, which feeds the signature evidence back into the learning tiers; the risk-keyword density (“risk_keyword_density”), the fraction of word tokens that are dangerous keywords; the method-and-path combination risk (“method_path_combo_risk”), which rises for sensitive paths such as “/admin” or “/login” and for write methods such as “PUT” or

“DELETE”; the count of special characters in the path itself (“path_special_chars”), which exposes traversal or injection embedded in the path rather than in the query; and the unknown-parameter ratio (“unknown_param_ratio”), the fraction of the request’s parameter names that were never observed on that exact endpoint during training—the feature that exposes the parameter-name-mutation attacks pervasive in the CSIC dataset.

The extraction procedure first decodes the URL to obtain a canonical payload and the encoding depth, then computes the three groups of features and concatenates them into a single vector. The procedure is deterministic and stateless except for the per-path vocabulary, which is learned once at training time and consulted by the unknown-parameter-ratio feature. Its steps are:

1. Concatenate the path and query string into the full URL.
2. Decode the full URL up to three times to recover the payload and to record the encoding depth.
3. Compute the structural features (length, path depth, POST flag, parameter count, risky-character ratio, and so on).
4. Compute the randomness-and-encoding features (decoded-URL entropy, maximum parameter-value entropy, encoding depth, and consecutive-encoding intensity).
5. Compute the semantic-and-contextual features (signature score, risk-keyword density, header-suspicion scores, and the unknown-parameter ratio against the learned vocabulary).
6. Concatenate the three groups into the final 22-dimensional vector and return it.



[Insert a screenshot of the “extract_features()” method and the “FEATURE_NAMES” list from “models/ai_detector.py” here.]

Figure 1.2: Source code of the 22-feature extraction routine (“extract_features” in “ai_detector.py”).

1.3 Tier 1 — Signature- and Rule-Based Detection

The first tier implements the misuse-detection paradigm. It is always active, requires no training data, and is the most trusted signal in the system because of its very high precision. Conceptually, the tier applies a battery of pattern matchers to the decoded

payload and reports which, if any, attack rule fired. The matchers cover the attack classes enumerated in Section 2.4 of Chapter 2: SQL injection, cross-site scripting, path traversal and local file inclusion, command injection, remote file inclusion, sensitive-file access, scanner user agents, HTTP method tampering, and parameter tampering.

1.3.1 The rule set

Each attack class is encoded as a pre-compiled, case-insensitive regular expression (or, for the two stateful rules, as a small procedure). The nine rules are summarised in Table 1.1; their exact definitions in the source code are shown in Figure 1.3.

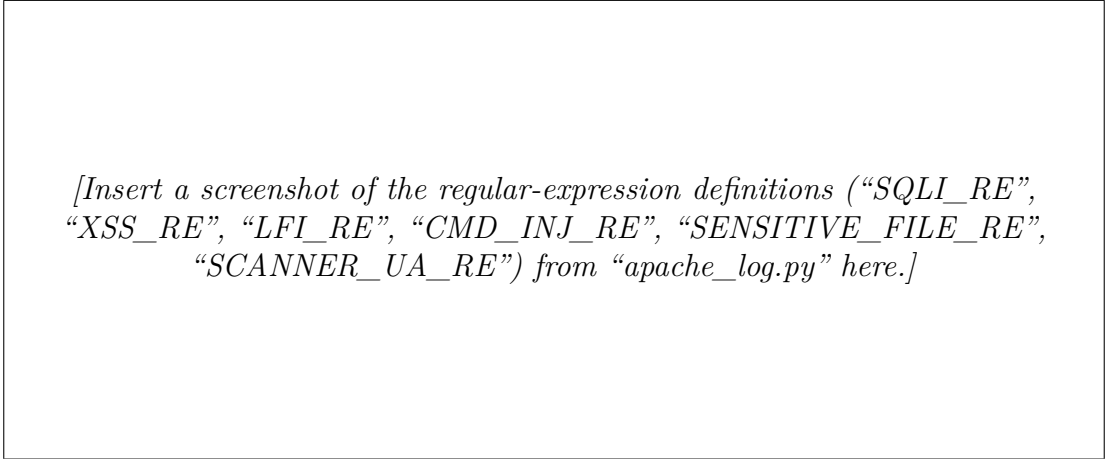
Table 1.1: The nine Tier 1 detection rules, their matching mechanism, and the attack class each targets.

Rule (identifier)	Matching mechanism (representative patterns)	Target attack
SQL_INJECTION	union select, select ...from, insert into, drop table, quoted tautologies such as 'OR'a'='a, comment markers -/#/*, xp_cmdshell	SQL injection
XSS	<script, javascript:, onerror=, onload=, eval(, <img src, alert(	Cross-site scripting
LFI_PATH_TRAVERSAL	../, ..\, /etc/passwd, /proc/self, C:\Windows, win.ini	Path traversal / LFI
COMMAND_INJECTION	;&&\$> before rm/cat/wget/curl/bash/nc/..., or back-tick command substitution	OS command injection
RFI	A parameter value beginning with http://, https://, or ftp://	Remote file inclusion
SENSITIVE_FILE_ACCESS	.env, .git/config, wp-config.php, .bak, .sql, /cgi-bin/, phpinfo.php	Config/backup file access
SCANNER_TOOL	User-Agent matching sqlmap, nikto, nmap, nessus, dirbuster, gobuster, acunetix, hydra	Automated scanners
HTTP_METHOD_TAMPERING	An unusual method (PUT, DELETE, PATCH, TRACE, CONNECT) on a non-API path	Method tampering
PARAM_TAMPERING	A parameter name absent from the per-path vocabulary learned during training	Parameter-name mutation

The first seven rules are purely pattern-based and stateless. Their design reflects a deliberate effort to balance coverage against false positives. For example, the SQL-injection expression was broadened beyond the classic “digit-equals-digit” tautology (“1=1”) to also match the *quoted, letter-based* tautologies such as “'OR'a'='a” that dominate the CSIC dataset, because the narrower pattern missed the majority of real CSIC payloads. The command-injection expression requires a shell metacharacter (a semicolon, pipe, double ampersand, dollar sign, or back-tick) *immediately before* a known binary name, which keeps the benign word “cat” in a product description from triggering the rule. The remote-file-inclusion rule inspects parameter *values* rather than the whole URL, so that a legitimate path containing “http” does not raise an alert.

The last two rules are stateful: they consult a *per-path parameter vocabulary* that the system learns at training time, recording for every endpoint the set of parameter names

that ever appeared on it. The method-tampering rule fires when one of the methods “PUT”, “DELETE”, “PATCH”, “TRACE”, or “CONNECT” is used on an endpoint that is not part of an API surface, since such methods are rarely legitimate on ordinary pages. The parameter-tampering rule fires when a request carries a parameter name that was never observed on that exact path during training; this is the mechanism that catches the frequent name-mutation attacks in CSIC, for example renaming “modo” to “modoA” or “precio” to “precioA”.



[Insert a screenshot of the regular-expression definitions (“SQLI_RE”, “XSS_RE”, “LFI_RE”, “CMD_INJ_RE”, “SENSITIVE_FILE_RE”, “SCANNER_UA_RE”) from “apache_log.py” here.]

Figure 1.3: Source code of the Tier 1 signature regular expressions in “apache_log.py”.

1.3.2 Rule evaluation

The crucial first step of the procedure is defensive decoding: the payload is decoded up to three times *before* any regular expression is applied, so that multi-layer encoded attacks such as a triple-encoded SQL tautology are normalised back to their canonical form and can be matched. Two of the rules are stateful and depend on the per-path vocabulary learned during training. The method-tampering rule fires when an unusual HTTP method (“PUT”, “DELETE”, “PATCH”, “TRACE”, “CONNECT”) is used on a non-API endpoint. The parameter-tampering rule fires when a request carries a parameter name that was never seen on that exact path during training, which is the mechanism that catches CSIC’s frequent name-mutation attacks (for example “modo” → “modoA”). The rule-evaluation procedure proceeds as follows:

1. Initialise an empty set of fired rule identifiers.
2. Form the candidate string by joining the path and the query string.
3. Decode the candidate string up to three times, stopping early if a decoding leaves it unchanged.
4. Apply each signature matcher (SQL injection, XSS, LFI/path traversal, command injection, remote file inclusion, sensitive-file access, scanner user agent) to the decoded string; for every match, add the corresponding identifier to the set.
5. Apply the method-tampering rule; if it fires, add its identifier.
6. Apply the parameter-tampering rule against the per-path vocabulary; if it fires, add its identifier.

7. Return the set of fired rule identifiers.

Figure 1.4 shows the implementation of this procedure in the “detect_rule_based” function.

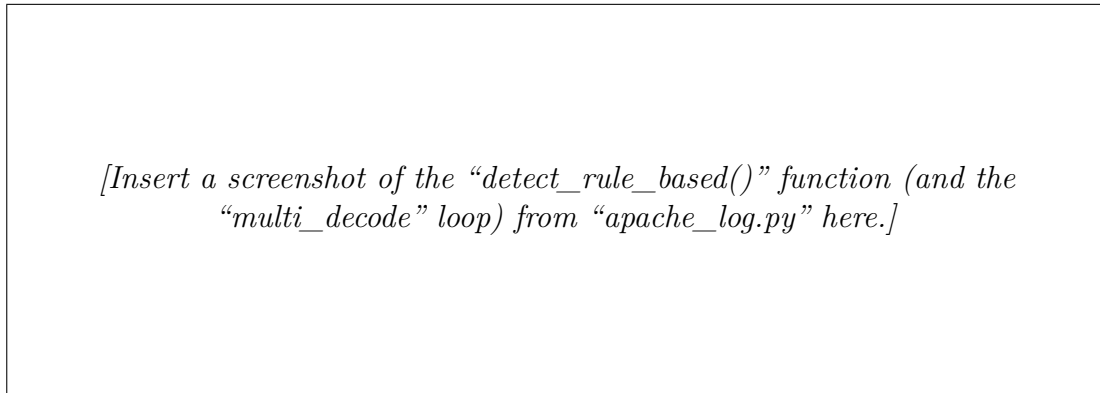


Figure 1.4: Source code of the Tier 1 rule-evaluation routine (“detect_rule_based” in “apache_log.py”).

As established experimentally in Chapter 2, this tier attains a precision of 99.61% but a recall of only 24.07%. In other words, when it fires it is almost always correct, but it stays silent on the majority of attacks. This profile makes it ideal as a high-confidence trigger within the consensus rule, but unsuitable as a standalone detector, which is the very reason the two learning tiers are introduced next.

1.4 Tier 2 — Supervised Detection with Random Forest

The second tier is a supervised classifier that learns attack patterns from labelled data. As justified in Chapter 2, the Random Forest is chosen as the primary model because of its robustness, its tolerance of unscaled heterogeneous features, and its ability to output a calibrated attack probability. The classifier is configured with 200 trees and balanced class weights and is trained on the labelled portion of the dataset using the same 22-feature vector as the other tiers.

Training comprises three conceptual steps. First, the per-path parameter vocabulary is built from the training logs so that the unknown-parameter-ratio feature is meaningful. Second, every labelled request is converted into a feature vector. Third, the Random Forest is fitted to the feature-label pairs. In detail:

1. Build the per-path parameter vocabulary from all training requests.
2. Extract the 22-feature vector for every labelled request, forming the feature matrix.
3. Instantiate a Random Forest with 200 trees and balanced class weights.
4. Fit the Random Forest to the feature matrix and the binary labels (1 for attack, 0 for benign).
5. Persist the trained model together with the vocabulary.

Recalling the mathematical formulation given in Chapter 2, the Random Forest trains $B = 200$ decision trees, each grown on a bootstrap sample of the training set and splitting nodes on a random subset of the 22 features using the Gini impurity $G = 1 - \sum_i p_i^2$ as the split criterion. The key property exploited here is that the forest produces not only a hard label but a calibrated probability, obtained by averaging the per-tree votes:

$$p_{\text{attack}}(x) = \frac{1}{B} \sum_{b=1}^B \mathbf{1}[T_b(x) = \text{attack}], \quad (1.2)$$

where $T_b(x)$ is the prediction of the b -th tree. This is exactly the quantity returned by the “predict_proba” method and consumed by the consensus rule. Figure 1.5 shows the training and inference code of the supervised tier.

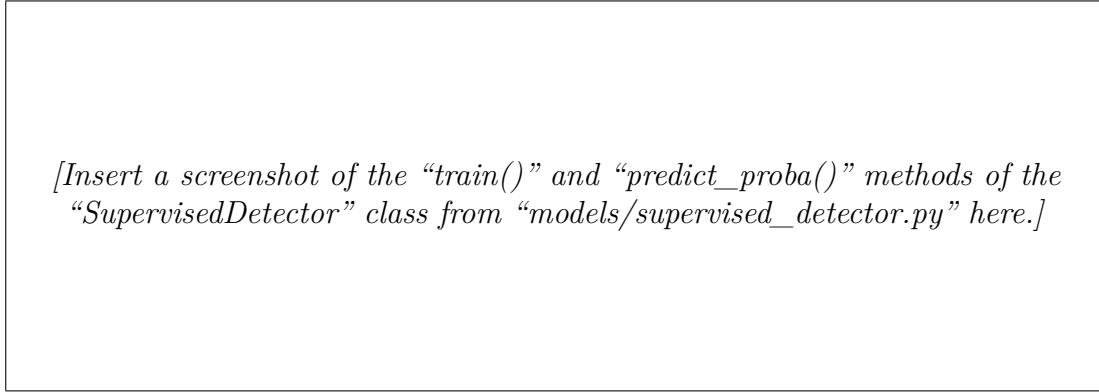


Figure 1.5: Source code of the supervised Random Forest tier (“supervised_detector.py”).

At inference time the model is queried for the probability p_{attack} that a request is malicious, computed by Equation (1.2); the binary Tier 2 hit is defined as $p_{\text{attack}} \geq 0.5$. Crucially, the consensus rule (Section 1.6) consumes the continuous probability rather than the thresholded decision, which lets it validate weaker anomaly signals at a lower probability cut-off. On the evaluation set this tier attains a precision of 94.80%, a recall of 94.76%, and an F1 score of 94.78%, the highest of any single model in the system. It therefore acts as the backbone of the hybrid: it both contributes a strong independent verdict and serves as the arbiter that confirms or rejects the anomaly tier’s findings. Its inherent limitation, shared by all supervised methods, is that it can only generalise from attack families present in the training data—hence the need for the third tier.

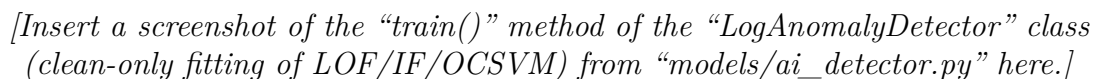
1.5 Tier 3 — Unsupervised One-Class Anomaly Detection

The third tier addresses the blind spot of supervised learning by detecting deviations from normal traffic without ever being shown an attack. As explained in Chapter 2, it follows the one-class learning principle: the model is trained *exclusively on clean traffic* (34,941 benign log lines) so that it learns the distribution of normal requests, and at inference any request that falls outside that distribution is flagged as anomalous. Three algorithms were implemented—Local Outlier Factor, Isolation Forest, and One-Class SVM—among which the Local Outlier Factor was selected as the operational model owing to its superior precision–recall balance.

Unlike the supervised tier, the training consumes no labels: the input is a corpus of attack-free requests. The features are extracted with the shared extractor, standardised so that distance-based density estimation is not dominated by high-magnitude features, and then passed to the one-class estimator. For the Local Outlier Factor the estimator is configured with twenty neighbours and in novelty mode, which allows it to score requests that were not part of the training corpus. The training steps are:

1. Build the per-path parameter vocabulary from the clean requests.
2. Extract the 22-feature vector for every clean request, forming the feature matrix.
3. Fit a standardisation transform on the feature matrix and apply it so that all features share a comparable scale.
4. Instantiate the Local Outlier Factor with twenty neighbours in novelty-detection mode.
5. Fit the model on the standardised clean features so that it learns the “normal” manifold.
6. Persist the model, the standardisation transform, and the vocabulary.

The mathematical formulation of the three one-class estimators (the local reachability density and Local Outlier Factor score, the isolation path length and anomaly score, and the One-Class SVM margin objective) was given in Chapter 2; Figure 1.6 shows how the chosen estimator is trained on clean data in the “train” method of the “LogAnomalyDetector” class.



[Insert a screenshot of the “train()” method of the “LogAnomalyDetector” class (clean-only fitting of LOF/IF/OCSVM) from “models/ai_detector.py” here.]

Figure 1.6: Source code of the unsupervised one-class training routine (“ai_detector.py”).

At inference the estimator returns an outlier indicator (-1 for an outlier, $+1$ for an inlier); the tier reports an anomaly precisely when the output is -1 . The decisive role of the clean-only training discipline is best illustrated by the experimental contrast: when the same model is mistakenly trained on a 50/50 mixture of attack and clean traffic, its F1 score collapses to about 33%, because the “normal” manifold becomes polluted with attack points and the notion of an outlier loses meaning. Trained correctly on clean data, the Local Outlier Factor reaches a precision of 88.28% and a recall of 95.13% (F1 = 91.57%). The high recall is exactly the property the hybrid needs from this tier: it catches attacks the supervised model misses, at the cost of a higher false-positive rate that the consensus rule is designed to suppress.

1.6 Smart Hybrid Consensus and Threat Scoring

The final stage fuses the three tier outputs into a single decision. A naive fusion would simply raise an alert whenever any tier fires (a logical OR); however, because the Local Outlier Factor alone produces a non-negligible number of false positives, such a rule would inflate the false-alarm rate. The proposed *smart consensus* instead lets the two high-precision signals fire on their own, while treating the anomaly tier as a tripwire that must be corroborated by the supervised model before it is trusted. Let R denote the set of Tier 1 rules that fired, p the Random Forest attack probability, and $a \in \{\text{true}, \text{false}\}$ the Tier 3 anomaly flag. A request is classified as an attack when

$$\text{attack} = (R \neq \emptyset) \vee (p \geq 0.5) \vee (a \wedge p \geq 0.3). \quad (1.3)$$

The three disjuncts of Equation (1.3) encode three independent ways to raise an alert. The first, $R \neq \emptyset$, fires when any signature matched. The second, $p \geq 0.5$, fires when the Random Forest is confident on its own. The third, $a \wedge p \geq 0.3$, is the key idea: a Tier 3 anomaly is trusted only when the Random Forest is at least mildly suspicious ($p \geq 0.3$), so an isolated anomaly that the Random Forest flatly rejects ($p < 0.3$) is not alerted, which is what suppresses the anomaly tier’s false positives.

Independently of the binary verdict, the system assigns a continuous threat score that drives the severity ranking shown to the analyst. The score aggregates the three signals with weights reflecting their trustworthiness:

$$\text{score} = 0.45 [R \neq \emptyset] + 0.40 p [p \geq 0.5] + 0.20 [a], \quad (1.4)$$

where the Iverson bracket $[\cdot]$ equals 1 when its condition holds and 0 otherwise. The signature tier therefore contributes a fixed 0.45 (the largest weight, as the most trusted signal), the supervised tier contributes 0.40 scaled by its own confidence p , and the anomaly tier contributes a fixed 0.20. The score is finally mapped to a severity level by

$$\text{level} = \begin{cases} \text{CRITICAL} & \text{if score} \geq 0.75, \\ \text{HIGH} & \text{if } 0.45 \leq \text{score} < 0.75, \\ \text{MEDIUM} & \text{if } 0.25 \leq \text{score} < 0.45, \\ \text{LOW} & \text{if score} < 0.25 \text{ and the verdict is an attack,} \\ \text{CLEAN} & \text{if the verdict is not an attack.} \end{cases} \quad (1.5)$$

Combining Equations (1.3) and (1.4) with the severity mapping of Equation (1.5), the decision procedure invoked for every request performs the following steps:

1. Initialise the threat score to zero.
2. Add 0.45 if any Tier 1 rule fired.
3. Add $0.40 \times p$ if the Random Forest probability p is at least 0.5.
4. Add 0.20 if the Tier 3 anomaly flag is set.
5. Compute the verdict using the smart consensus rule of Equation (1.3).
6. If the verdict is “not an attack”, set the severity to CLEAN; otherwise map the score to CRITICAL, HIGH, MEDIUM, or LOW using the thresholds of Equation (1.5).

[Insert a screenshot of the smart-consensus decision logic (the “should_alert”/“is_attack” expression and the threat-score accumulation) from “apache_log.py” here.]

Figure 1.7: Source code of the smart hybrid consensus and threat-scoring logic in “apache_log.py”.

7. Return the verdict, the threat score, and the severity level.

Figure 1.7 shows the implementation of this fusion rule, where the boolean verdict of Equation (1.3) is computed in a single expression over the three tier outputs.

A practical advantage of this design is its *explainability*: because the verdict is a transparent combination of three named signals, every alert can be annotated with the tiers that contributed to it (for example “REGEX(SQL_INJECTION) + SUPERVISED(RF, p=0.98)”), which helps the analyst triage quickly. As reported for the full pipeline, the smart consensus achieves a precision of 91.19%, a recall of 97.62%, an F1 of 94.30%, and an F2 of 96.26%—the highest F2 in the system, confirming that the fusion meets the IDS priority of minimising missed attacks while keeping false alarms manageable.

1.7 Operating Modes: Static Scan and Real-time Monitoring

The same three-tier pipeline described above is exposed through two operating modes, so that the system can be used both for the offline analysis of historical logs and for live protection of a running web server. Both modes share exactly the same parsing, feature extraction, three-tier evaluation, and smart-consensus decision; they differ only in how log lines enter the system and how the results are delivered.

1.7.1 Static file scan

The static-scan mode is intended for the offline, batch analysis of a log file that has already been collected—for example for a forensic audit of historical traffic or for the reproducible evaluation of the models. It is invoked from the command line by passing the “scan” action, the path to the log file, and the desired Tier-3 model, as in the command

```
python apache_log.py scan <logfile> lof
```

The system then reads the file line by line and, for every request, runs the full pipeline and produces two output files in JSON Lines format. The first, “scan_results.jsonl”, contains one record *per log line*—both clean and attack—each carrying the verdict, the threat score, the severity level, and the list of tiers that contributed, so that the outcome of

every request can be audited. The second, “alerts.jsonl”, contains only the lines judged to be attacks, in a compact alert format. At the end of the scan the system prints a summary that reports the total number of lines processed, the split between clean and attack, and the distribution of alerts across the CRITICAL, HIGH, MEDIUM, and LOW severity levels. A representative record from “scan_results.jsonl” is shown below:

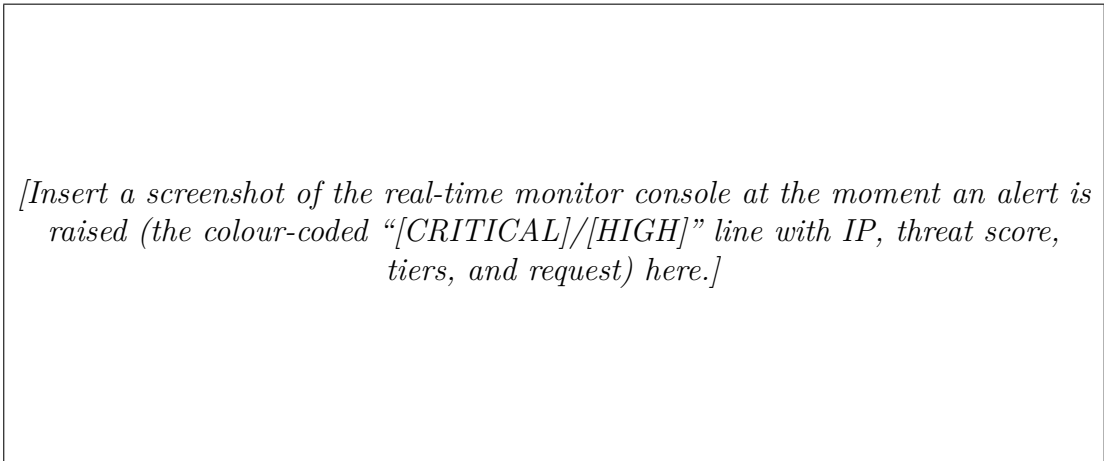
```
{ "raw_url":  "/login?POST_BODY=user...", "verdict":  "ATTACK",  
  "level":    "HIGH", "score":  0.94, "rule_ids":  [],  
  "sup_prob": 0.98, "lof_hit": true }
```

1.7.2 Real-time monitoring

The real-time monitoring mode turns the system into a live sensor that analyses requests as they arrive. It reads log lines from standard input, which lets it be attached directly to a growing server log with a single shell pipeline, for example

```
tail -F /var/log/apache2/access.log |  
  python apache_log.py monitor /dev/null lof
```

Every incoming line is passed through the identical three-tier consensus, and whenever a request is judged to be an attack the system emits a colour-coded alert on the console—the colour reflecting the severity level—showing the timestamp, the source IP address, the aggregated threat score, the contributing tiers, and the offending request. In parallel, each alert is appended as a structured record to “runtime/monitor_alerts.jsonl”. A companion dashboard, implemented with Streamlit, continuously tails this file and presents the information visually: key-performance-indicator cards (such as the detection rate and the false-positive rate), a live feed of the most recent alerts, the most active source IP addresses, and the distribution of alerts by threat level. Together, the console output and the dashboard give an analyst an immediate, continuously updated picture of the attacks hitting the server. Figure 1.8 shows the console of the real-time monitor at the moment an attack is detected, and Figure 1.9 shows the corresponding view on the dashboard.



[Insert a screenshot of the real-time monitor console at the moment an alert is raised (the colour-coded “[CRITICAL]/[HIGH]” line with IP, threat score, tiers, and request) here.]

Figure 1.8: Real-time monitoring console raising an alert the moment an attack is detected.

[Insert a screenshot of the Streamlit dashboard (KPI cards, live alert feed, top source IPs, threat-level distribution) here.]

Figure 1.9: Real-time monitoring dashboard summarising the live alert feed and key performance indicators.

1.8 Chapter Summary

This chapter has presented the proposed solution in full. Section 1.1 gave an overview of the three-tier hybrid pipeline and described its data flow (Figure 1.1) together with the overall scanning procedure. The subsequent sections detailed each stage: request parsing with multi-layer decoding and the construction of the shared 22-feature vector (Section 1.2); the always-on signature and rule tier with its defensive decoding and stateful tampering rules (Section 1.3); the supervised Random Forest tier that supplies a calibrated attack probability (Section 1.4); and the unsupervised one-class anomaly tier trained exclusively on clean traffic (Section 1.5). Finally, Section 1.6 introduced the smart consensus rule (Equation (1.3)) and the weighted threat score (Equation (1.4)), which together turn three complementary opinions into a single, explainable, and recall-oriented verdict. Finally, Section 1.7 described the two operating modes that expose this pipeline in practice: the static file scan for offline, batch analysis of historical logs, and the real-time monitor with its live dashboard for protecting a running server. The next chapter evaluates this solution empirically, comparing the individual tiers and the hybrid configurations on the benchmark dataset.